An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary recursively moves items into ordered subproblems, extending the classical framework in which items move down a binary tree.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys. A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them. Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

The classical sorting lower bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [8].

Deterministic adversaries from graph entropy. Kahn and Kim [10] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex

optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Randomized approximation of the number of linear extensions. Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order P, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [7]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [9] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by P. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most ε, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most n^{-4}. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in n. Kaplan, Zamir,

and Zwick [5, Section 7] explicitly ask for an adversary forcing $(1 - o(1))n \log n$ comparisons. Our main theorem answers this question.

Theorem 1.1. *There is a deterministic legal adversary, initialized using only n, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

comparisons.

Merge sort uses at most $n \lceil \log n \rceil$ comparisons, so this bound is asymptotically optimal. The derandomization uses a restricted recursive representation of the remaining orders. The structure of these regions allows the adversary to calculate the required volume estimates deterministically in polynomial time. Its running-time guarantee is asymptotic and involves very large parameter thresholds.

Section 2 reviews the classical tree adversaries and the construction of Kaplan, Zamir, and Zwick. Section 3 gives an overview of our state, comparison operation, and refinement operation. Section 4 specifies the complete algorithm and its parameters. Section 5 proves that its answers remain consistent with a total order. Section 6 justifies the local calculations. Sections 7 and 8 establish the comparison bound, and Section 9 proves polynomial running time. The geometric and numerical proofs used by the local calculation are in the appendices. The manuscript source is available online.[1]

2 Related Work

2.1 Binary-tree adversaries

Atallah and Kosaraju [1] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [2] on searching after partial sorting, Richards and Vaidya [3] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [4] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and

[1] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

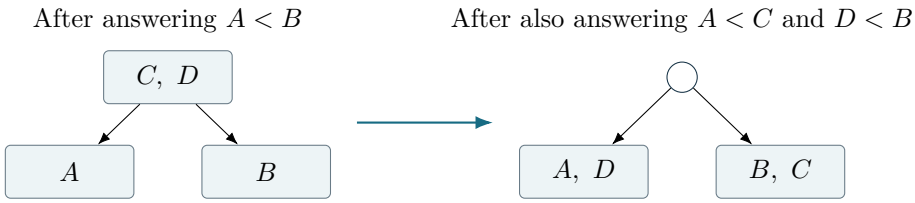


Figure 1: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2} n \log n$ comparisons are necessary [4, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

2.2 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [6] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i+1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child i. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j-1]$ or $[i+1, j]$. Figure 2 shows this interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

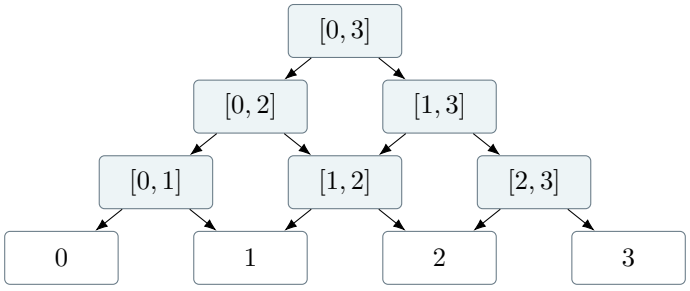


Figure 2: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [6, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree nodes and intermediate nodes representing three children may store pairs; intermediate nodes representing two children store only unpaired items. Comparing two unpaired items A, B at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing A and C, one permitted response

is $A < C$: move A into child 0 and B into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [6, Table 11].

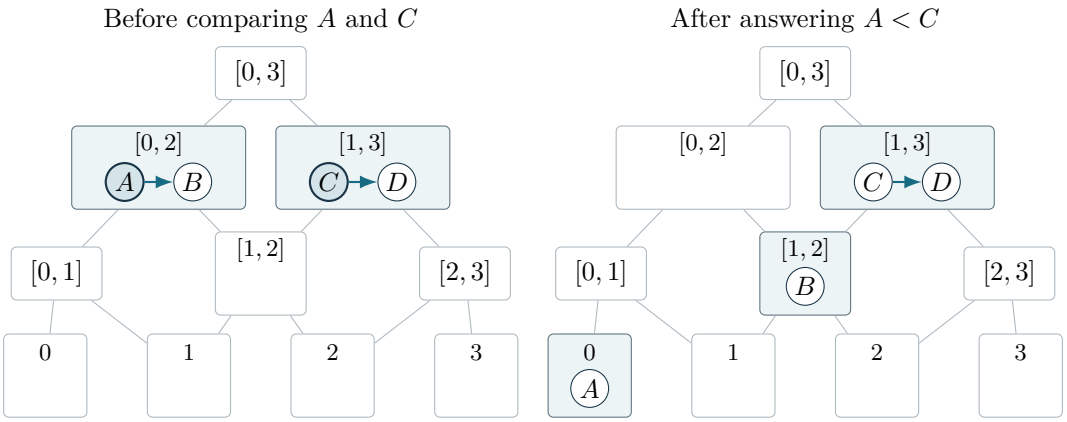


Figure 3: A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [6, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to A and C.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then A can enter only child 0 or 1, and B can enter only child 1 or 2. The pair's order thus narrows each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [6, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64} n \log n$ comparisons, improving the earlier $\frac{1}{2} n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [6, Section 6].

3 Overview of our adversary

We now give an overview of the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

Kaplan, Zamir, and Zwick [6] retain disjoint ordered pairs within a node. Our adversary maintains a rooted tree with a DAG at each node, so several retained comparisons can involve the same item. For example, it can keep $a < b$ and $a < c$ together while leaving the order of b and c undecided. This lets it represent richer partial orders before moving items further down the tree.

The computational challenge is to use these richer orders without having to approximately count the linear extensions of an entire DAG. We periodically simplify a node's DAG by removing vertices and moving their items into child nodes, creating children when needed. Constraints at the parent preserve the comparisons involving the moved items, while their order within a child can remain unresolved. These simplifications allow comparison answers to be chosen by deterministic calculations on small neighborhoods in the DAG. The analysis must show that the resulting loss of possible orders is small enough to retain the optimal comparison bound.

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. Figure 4 shows an example.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and X_x is a symbolic variable for its position among the items at that node. The fields a_x, b_x restrict this variable to the open interval $(a_x, b_x) \subseteq (0, 1)$. We call an item *free* at its node while its position remains variable.

In a node's DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. A feasible assignment of positions at that node satisfies both its interval bounds and its arc inequalities. Each answered comparison between free items contributes at most one arc.

Initially the tree consists of just the root, containing all records with bounds $a_x = 0$ and $b_x = 1$. The position variables X_x are unassigned, and the root's DAG has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position variables again range over $(0, 1)$, and its DAG and interval bounds describe the possible orders inside this block.

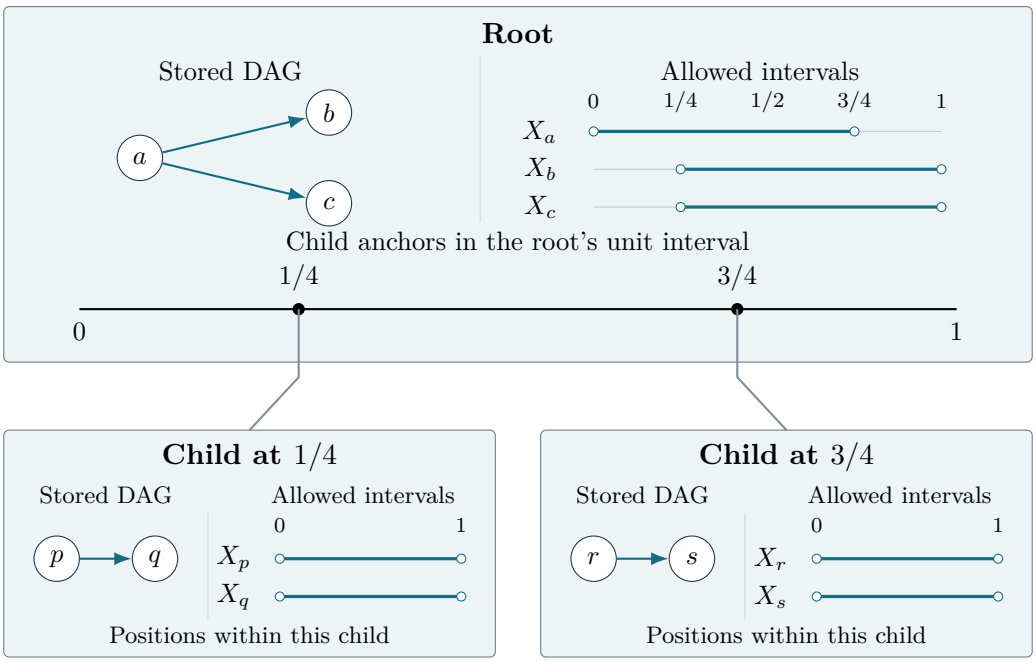


Figure 4: A state of the adversary. Each large box is a tree node, containing a DAG and one open interval for each of its items.

In the root of Figure 4, the DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. Those answers are also kept in a separate graph on the original item labels.

3.2 Answering a comparison

Algorithm 1 shows the algorithm the adversary uses to answer a comparison by the sorter.

To compare A and B, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. In Figure 4, comparing p with q descends into the left child; comparing p with r uses the anchors $1/4$ and $3/4$; and comparing a with p compares X_a with $1/4$ at the root.

Two distinct anchors already determine an answer. Interval bounds or a directed path in the node's DAG may also determine it. Otherwise, let p be the probability that A precedes B when positions at this node are assigned uniformly subject to its interval bounds and DAG inequalities. The adversary computes a deterministic estimate $\widehat{p}$ of this probability. It answers $A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise. The calculation uses only a small neighborhood of the queried items in the DAG, together with interval bounds implied by the full graph. Section 6 gives the calculation and proves its accuracy.

An answer between two free items adds an arc to their DAG; an answer between a free item and an anchor tightens the item's interval.

Before calculating $\widehat{p}$, we propagate all interval bounds implied by the DAG. This gives each item's *effective interval*, its full range of feasible positions. For the probability estimate to be accurate, degrees and directed paths must stay below prescribed limits, and at least one queried free item must have an effective interval of sufficient width.

We maintain these conditions by moving items into children. Such a move is called a *refinement*, denoted `Refine`(x) for item x. Section 3.3 describes how it preserves earlier answers. If all queried free intervals are too narrow, refine one of those items before answering. After answering, we restore the graph bounds by refining vertices with too many neighbors, then refining every vertex of any selected path that is too long. All selections use fixed tie-breaking rules.

Algorithm 1: `Compare`(A, B): overview of a comparison query.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.
while *true* **do**
  $v \leftarrow$ the root
  while *A and B lie in the same child subtree of v* **do**
    $v \leftarrow$ that child
  if *A and B lie in different child subtrees of v* **then**
    answer $\leftarrow$ the left-to-right order of their child anchors
    return *answer*
  Propagate interval bounds through the DAG at v
  if *interval bounds or a directed path in the DAG determine the answer* **then**
    return the determined answer
  if *at least one queried free interval is wide enough* **then**
    break
  `Refine`(one of the queried free items)
$\widehat{p} \leftarrow$ deterministic estimate of $\mathbb{P}(A < B)$ at v
answer $\leftarrow A < B$ if $\widehat{p} \geq 1/2$, and $B < A$ otherwise
Store the arc or interval bound for *answer* at v
Propagate interval bounds through the DAG at v
while *a vertex at v has too many neighbors* **do**
  `Refine`(such a vertex)
while *the DAG at v has a directed path that is too long* **do**
  Select such a path and `Refine` all its vertices
return *answer*

Refining a queried item restarts the search from the root because the node that handles the query may change. In the last loop, the whole selected path is refined before another is selected.

3.3 Moving an item into a child

To carry out `Refine`(x), first propagate all interval bounds implied by the DAG. Choose a point c inside x's effective interval and fix its parent position at $X_x = c$.

The incident arcs then become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, retain the bounds, and propagate them through the remaining DAG. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 5 illustrates reuse of an existing child.

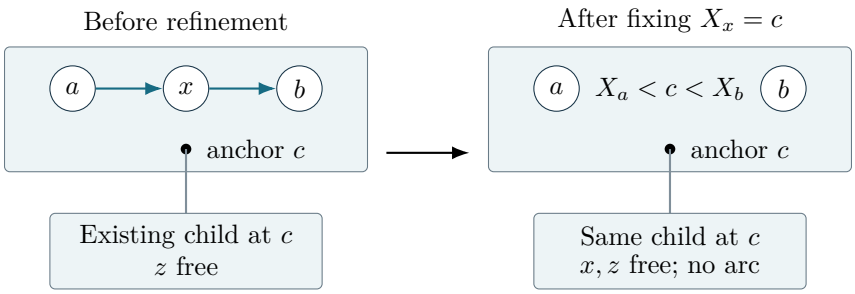


Figure 5: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of x within that block stays open. Lemma 5.2 proves that this operation preserves all answered comparisons. The deterministic rule for choosing c is given in Section 6.3; its role is to control how many possible orders refinement discards.

The proofs must account for the orders discarded by refinements as well as by comparison answers. Section 5 establishes legality; Sections 7 and 8 bound the total loss and prove the comparison lower bound. Section 9 shows that the local calculations and all updates take polynomial time.

4 The complete algorithm

We first describe what a state means and how an item moves from a parent to its child. We then give the comparison rule and the conditions that trigger these moves. The numerical parameters are collected at the end of the section, after their roles have been explained.

4.1 A tree of groups, each with its own coordinates

The state is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG) on the items stored there. We call this DAG the node's *active graph*.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and the symbolic variable X_x describes its position among the items at that node. The record's bounds impose $a_x < X_x < b_x$, with $(a_x, b_x) \subseteq (0, 1)$. We call the item *free* while its position remains variable. An arc $x \to y$ in the node's active graph records the requirement $X_x < X_y$.

A child occupies a fixed position $c \in (0, 1)$ in its parent. This position, called its *anchor*, determines how the child's items compare with items at the parent: $X_x < c$ places x before every item in the child subtree, and $X_x > c$ places it after every item in that subtree. If two children have anchors $c < d$, every item in the subtree at c precedes every item in the subtree at d. Each child subtree thus occupies a consecutive block in the represented order. Its internal order is determined recursively, using position variables in $(0, 1)$ at the child. Different children of a node have different anchors. The allowed assignments at each node must have positive volume; Section 5 proves that every operation preserves this property.

To read an order from the state, choose allowed values of the free coordinates at each node. At the root, list its free items and child anchors from left to right. Replace each anchor by the list obtained in its child, and continue recursively. We choose values avoiding ties; these form all but a set of volume zero. Every item appears exactly once in the resulting list. The collection of feasible assignments therefore specifies the family of total orders represented by the state.

Initially the tree has only a root. All n items are free there, with records $(x, X_x, 0, 1)$ and unassigned position variables X_x. There are no arcs, so the feasible assignments form $(0,1)^n$. Assignments with distinct positions represent all $n!$ total orders. The algorithm will ensure that every order it represents is consistent with all the comparisons it has answered. It may discard additional orders when it changes the representation; the lower-bound proof will account for that loss.

4.2 Refinement: fixing the parent position

The basic operation moves an item x into a child. Choose a position c strictly inside the interval of values still possible for X_x. Replace its parent coordinate by the constant c. The inequality imposed by each arc incident to x then tightens an interval bound on its other endpoint:

$$y \to x \implies b_y \leftarrow \min(b_y, c), \qquad x \to z \implies a_z \leftarrow \max(a_z, c).$$

Retain every other bound already stored in the parent. Remove x and these arcs, then propagate the new bounds through the remaining graph (the operation `Tighten` defined below). Now put x in the child anchored at c, creating that child if necessary. It receives a new coordinate $U_x \in (0,1)$ with no incident arcs. None of the child's previous intervals or arcs changes. We call this operation a *refinement*. Thereafter we may write X_x for its new coordinate, since its old coordinate is no longer a variable.

The fixed parent position specifies where the child block occurs among the other parent items. The new variable specifies where x occurs within that block. Its fresh interval $(0,1)$ allows every placement of x among the child's existing items.

For example, suppose the root contains A, B, C, with the sole requirement $A < B$. If we refine A at $1/2$, then B's parent interval becomes $(1/2, 1)$ and A receives a unit coordinate in the child at $1/2$. If we next refine C at $1/2$, the same child contains two free items, A and C, with no comparison between them. Both orders

$$A < C < B \qquad \text{and} \qquad C < A < B$$

are represented. The comparison $A < B$ has been preserved. The order of A and C remains open. The additional relation $C < B$ was not queried; this example also shows why refinements must be paid for in the proof. The item B cannot enter this child, since $1/2$ is outside its allowed open interval.

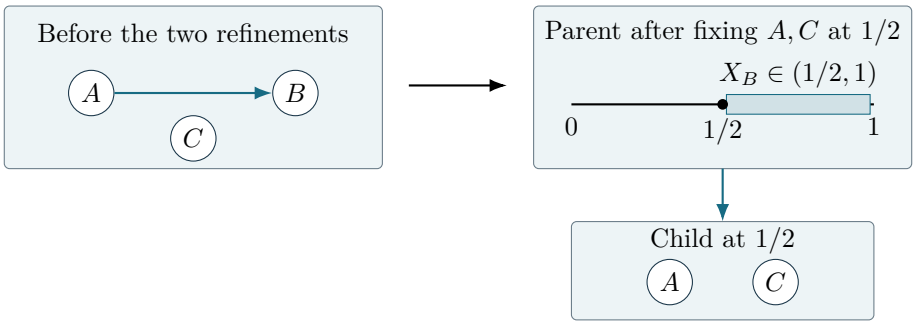


Figure 6: Refining two incomparable items at the same position leaves their mutual order unresolved in a shared child. Their new positions each range over $(0,1)$ within the child. The parent's bound on B puts the whole child before B.

4.3 Where a comparison is answered

To compare distinct original labels A and B, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or

when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. There are three cases.

1. Two different anchors determine the answer from their left-to-right order.
2. For a free coordinate X_x and an anchor c, decide whether $X_x < c$ or $X_x > c$.
3. For two free coordinates, decide whether $X_A < X_B$ or $X_B < X_A$.

In Figure 6, comparing A with C descends into their common child. Comparing either one with B stays at the root and compares $1/2$ with X_B; the stored bound already puts B to the right. More generally, the order of the child anchors determines every comparison between their respective subtrees.

If effective interval bounds or a directed path already determine a query, return that answer. The stored state then remains unchanged. Otherwise use the following estimate.

For either case involving free coordinates, let

$$K_v = \{X : a_x < X_x < b_x \text{ for all free } x, \quad X_x < X_y \text{ for every stored arc } x \to y\} \tag{1}$$

be the region of allowed assignments at this node, and let $V_v = \mathrm{vol}(K_v)$. The uniform distribution on K_v gives equal weight to equal-volume parts. For example, the fraction compatible with $A < B$ is

$$p = \mathbb{P}(X_A < X_B) = \frac{\mathrm{vol}(K_v \cap \{X_A < X_B\})}{V_v}.$$

To choose an answer, estimate this ratio. Given $\widehat{p}$ with $|\widehat{p} - p| \le \eta$, choose $A < B$ if $\widehat{p} \ge 1/2$, and $B < A$ otherwise. Either choice retains at least $1/2 - \eta$ of the old volume. The adversary calculates $\widehat{p}$ deterministically from the node's volume constraints.

Store the chosen answer as an arc when both coordinates are free. Each stored arc corresponds to an answered comparison and appears once, including when that comparison is repeated. For a free item x and an anchor c, estimate $\mathbb{P}(X_x < c)$ instead and store the chosen upper or lower interval bound. Translate that direction back to the queried labels, reversing it if x is the second label.

To obtain an estimate, the algorithm keeps a neighborhood of the queried items in the graph, with the interval bounds already implied by the full graph. It initializes temporary numerical positions at their lower bounds, then repeatedly updates each position within the interval permitted by its neighbors. Each update uses a value from a prescribed finite set. The algorithm enumerates every sequence of these choices and returns the fraction for which the requested comparison holds. Section 6 gives the update rule, the finite sets, and the number of repetitions, and proves the accuracy of this average.

The finite calculation may produce equal positions. We estimate the one event $A < B$ and use $1 - \widehat{p}$ for $B < A$. The two true probabilities sum to one, so both estimates have the same error bound. The test $\widehat{p} \ge 1/2$ therefore chooses an answer retaining at least $1/2 - \eta$ of the feasible volume.

4.4 The three reasons for refinement

The estimate just described is available when each vertex has few neighbors, every directed path is short, and at least one queried coordinate has an interval of adequate width. We arrange these conditions as follows. Fix an integer $j \ge 6$, put $k = 2^j$, and set the error $\eta = 1/(100k)$. Use a path threshold H and a width threshold $1/R$, with values given below.

First we include every interval bound implied by the arc inequalities. If $y \to x$ and $X_y > a_y$, then also $X_x > a_y$; an upper bound propagates in the reverse direction. Writing $y \preceq x$ when $y = x$ or a directed path leads from y to x, the resulting interval is

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x. \tag{2}$$

We call (ℓ_x, r_x) the *effective interval*. The operation `Tighten` sets $a_x \leftarrow \ell_x$ and $b_x \leftarrow r_x$ for every free item x. It is performed after any change to the arcs or intervals, and before a width test or probability calculation. These bounds remain stored even if the arcs that implied them are later removed. Tightening does not discard any allowed assignment.

There are exactly three reasons to refine an item.

1. **A comparison needs a wider coordinate.** Call a coordinate *wide* when $W_x \geq 1/R$, and *narrow* otherwise. A comparison estimate needs one wide free coordinate. If all free coordinates in the requested comparison are narrow, refine one of them before answering. It receives an unconstrained unit coordinate in its child. The reason one wide coordinate is enough is explained in Section 6.
2. **A vertex has too many neighbors.** After an answer, if a vertex has at least k neighbors, refine it. This removes all arcs incident to it from the parent graph.
3. **A directed path is too long.** After the degree condition has been restored, select any directed path with H vertices and refine all its items. Complete this selected list before selecting another path. Repeat until no such path remains.

We use the word *repair* for an operation that restores one of these conditions. A path repair consists of the entire selected list. The analysis charges for each completed path repair as a whole.

4.5 How the fixed position is chosen

Anchors are midpoints obtained by repeatedly halving the unit interval. After m halvings the cells have length 2^{-m}, and their midpoints are

$$c = (i + \tfrac{1}{2})2^{-m}, \qquad 0 \leq i < 2^m. \tag{3}$$

We call m the anchor's *level*. For example, $1/2$ has level 0, and $1/4, 3/4$ have level 1. There are 2^m possible anchors at level m. An anchor has only one level, since the reduced denominator of its midpoint is 2^{m+1}. Thus repeated use of a position also reuses its level. This level measures resolution within one parent interval, independently of that parent's depth in the tree. Every refinement moves an item down exactly one tree edge.

For a wide item, we find a midpoint with a substantial fraction of the allowed assignments on either side. Start with a search cell $[0, 1]$. At its midpoint c, estimate $F_x(c) = \mathbb{P}(X_x < c)$ to error η. If the estimate is below $1/4$, search the right half; if it is above $3/4$, search the left half; otherwise accept c. After a rejected test, update the search cell and test its midpoint. Every test uses the same stored intervals, arcs, and feasible region K_v.

For a narrow item, no probability calculation is needed. Choose the unique cell length $b = 2^{-m}$ satisfying $W_x/8 < b \leq W_x/4$. Among the midpoints of that grid, use the one nearest to $(\ell_x + r_x)/2$, taking the smaller one on a tie. It lies near the center of the effective interval.

Both rules choose an interior point. Section 6 proves the property needed for the lower bound: fixing a level-m point recovers a volume factor whose logarithm is at least $(1 - 1/k)m - 4$.

4.6 The mechanical rules

Every answer is also recorded in a separate graph on the original labels, before any restoration or return. This routine update is implicit in the pseudocode. We call the comparison history the *transcript* and the graph the *public comparison graph*. Its arcs are never counted as part of a node's active graph. In Section 9 we use it to finish with a compatible fixed order after a prescribed number of queries.

Resolve choices between eligible items by label order. Select a directed path by the lexicographically first eligible label sequence. Degree counts neighbors regardless of arc direction; the length of a directed path counts its vertices.

Algorithm 2: `Restore`(v): restore the degree and path-length bounds.

Input: The node v that has just answered a query.
Output: The DAG at v has degree below k and every directed path has fewer than H vertices.

```
while the DAG at v has a vertex x of degree at least k do
    Refine(x)
while the DAG at v contains a directed path on H vertices do
    P ← the selected path
    foreach x ∈ P in label order do
        Refine(x)
```

Algorithm 3: `Compare`(A, B): the complete comparison procedure.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.

```
while true do
    v ← the root
    while A and B lie in the same child subtree of v do
        v ← that child
    if A and B lie in different child subtrees of v then
        answer ← the left-to-right order of their child anchors
        return answer
    Tighten(v)
    if interval bounds or a directed path in the DAG determine the answer then
        return the determined answer
    if some queried free coordinate is wide then
        break
    Refine(the selected queried free item)
p̂ ← an estimate of ℙ(A < B) to error η
if p̂ ≥ 1/2 then
    answer ← (A < B)
else
    answer ← (B < A)
Store the arc or interval bound for answer at v
Tighten(v)
Restore(v)
return answer
```

After a preparation refinement, Algorithm 3 searches again from the root because moving an item may change the node where the comparison is answered. There are at most two preparation refinements: once either queried item has received a fresh wide coordinate, it cannot need preparation again before this answer (Lemma 5.4).

For completeness, the parameters used by these rules are

$$j \geq 6, \qquad k = 2^j, \qquad \eta = \frac{1}{100k}, \qquad H = 2^k, \\ B = (k+1)^{2H+1}, \qquad R = 2^{2kB}. \tag{4}$$

Here B bounds how many other vertices can be joined to one vertex by a directed path in either direction during a repair. Section 5 proves that bound. The value of H makes a long path expensive enough to pay for refining all its items. The value of R makes a narrow interval small enough to pay for refining its item. The bounds needed for these claims are proved in Sections 6–8. Finally, Section 9 chooses j from n at initialization and keeps it fixed throughout the interaction. Its slow growth makes the whole algorithm polynomial in bit complexity.

5 Consistency of the answers and graph bounds

The state describes a family of total orders. We must show that this family never becomes empty and that every order in it respects the answers already given. We must also show that the operations keep the graphs within the bounds required by the probability calculations.

5.1 Preserving the represented orders

Recall how a state represents an order. Choose feasible coordinates in each node, order its free items and child anchors from left to right, and replace each anchor by the order represented inside that child. Each child subtree therefore forms a consecutive block. We exclude equalities between free coordinates, or between a free coordinate and an anchor; these occur on a finite union of sets of volume zero.

We maintain the following invariant:

> Every represented total order respects every comparison already answered.

The state may discard some orders that still respect the answers. Legality requires only that at least one remains. Positive volume in every node will ensure this.

There are two graphs to keep distinct. The public comparison graph has the original labels as vertices and records every answer. A directed path in it is an order relation implied by the transcript. An active graph belongs to one node and contains only its stored comparisons between currently free items. After an active arc is removed, the recursive state continues to enforce its corresponding public answer.

Lemma 5.1 (Effective intervals and sections)**.** *For an active DAG, tightening leaves the feasible region unchanged. The region has positive volume if and only if every effective width is positive. Moreover, fixing $X_x = c$ at any $c \in (\ell_x, r_x)$ leaves a positive-volume section in the remaining coordinates.*

Proof. If $y \neq x$ precedes x on a directed path, every feasible assignment satisfies $X_x > X_y > a_y$; the stored bound $X_x > a_x$ covers $y = x$. Taking the maximum over these bounds gives $X_x > \ell_x$. The upper-bound argument is the reverse. Thus all bounds introduced by (2) already hold, so tightening changes no feasible assignment.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Along an arc $x \to y$, the definitions give $\ell_x \leq \ell_y$ and $r_x \leq r_y$. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Increasing θ_x to θ_y strictly increases this value because $r_x > \ell_x$. Replacing its endpoints by the larger endpoints ℓ_y, r_y cannot decrease it. Hence $X_x < X_y$. All coordinates are inside their intervals and all arc inequalities hold strictly, so a sufficiently small open neighborhood of this assignment is feasible.

Now fix $X_x = c$ and delete its coordinate. The inequalities imposed by its incident arcs become upper bounds c at its in-neighbors and lower bounds c at its out-neighbors. Upper bounds

propagate only backward along arcs, and lower bounds only forward. Since all previous effective bounds remain stored, the new effective intervals are therefore

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases}$$

The first two cases are disjoint because the graph is acyclic. A predecessor retains positive width: $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too: $r_y > \ell_y$ and $c < r_x \leq r_y$. All other widths stay positive. The criterion just proved, applied to the remaining graph, shows that the section has positive volume. □

Lemma 5.2 (Refinement and child admission)**.** *Suppose the node regions have positive volume and the order invariant holds. Refining x at an interior anchor c preserves both properties, including when the child at c already exists. The public transcript implies no relation between x and any label already in that child subtree.*

Proof. Consider first an existing child at c, and denote its label set by S. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma 5.1. Also choose feasible assignments in the other nodes.

There is a positive distance from c to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at x are strict. Thus, with everything else fixed, x can be moved slightly to either side of c while keeping the old state feasible. The resulting two represented orders place x immediately before or immediately after the entire block S. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the public comparison graph.

Now take any order represented after the refinement. Deleting x from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except x. The new order inserts x somewhere into their child block. No answered comparison joins x to a label in S, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item x is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at c, the same argument uses an empty block: the new singleton child places x where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

5.2 The volume gained by refinement

Let $V_v = \mathrm{vol}(K_v)$ be the volume of node v's feasible region, as in the comparison rule. Define

$$Z = \prod_v V_v. \tag{5}$$

Coordinates in different nodes are separate variables. Their feasible regions form a Cartesian product, so Z is its volume. There is exactly one free coordinate per original item, giving total dimension n. A node with no free coordinates contributes the zero-dimensional volume one. Initially $Z = 1$; whenever all node regions are positive, $0 < Z \leq 1$ because each is contained in its unit cube.

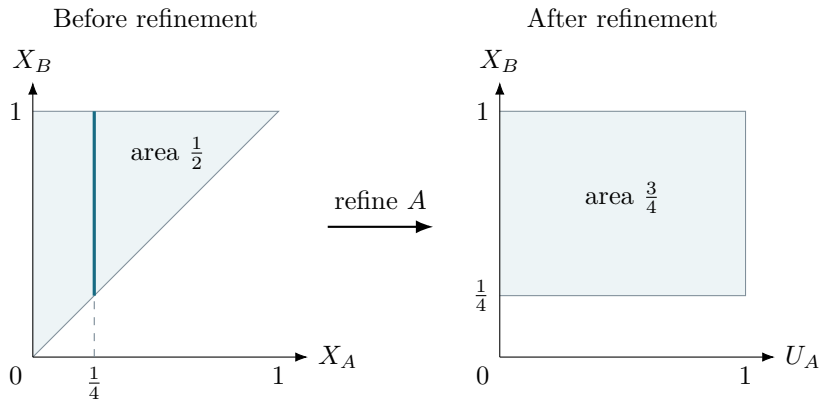


Figure 7: The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

Suppose x is free at a node v with d free coordinates. Write $S_x(c)$ for the $(d-1)$-dimensional volume of the section obtained by fixing $X_x = c$. Slicing K_v along the x coordinate gives $V_v = \int S_x(t)\,dt$. Thus the density of X_x under the uniform volume measure is

$$f_x(c) = \frac{S_x(c)}{V_v}.$$

This density is exactly the factor by which refinement changes Z.

Lemma 5.3 (Refinement identity)**.** *Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{6}$$

Proof. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0,1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0,1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

For example, the parent region $0 < X_A < X_B < 1$ has area 1/2. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length 3/4. The new child coordinate $U_A \in (0,1)$ turns this into a rectangle of area 3/4, so refinement multiplies Z by 3/2; see Figure 7. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

5.3 Bounds during a comparison call

We next check how the graph conditions evolve. The numerical lemmas of Section 6 will guarantee interior refinement positions and positive-volume answers whenever the current region is positive and has the stated graph bounds. We first give the corresponding bounds on graph changes, and then combine the facts in an induction.

Lemma 5.4 (Query preparation)**.** *Before selecting its answer,* Compare *performs at most two refinements. Each selected coordinate is narrow, and preparation preserves the graph bounds.*

Proof. A preparation refines one of the two original queried items. Its new coordinate is isolated and has interval $(0, 1)$. Before an answer is stored, any further preparation only removes a parent coordinate and adds an isolated coordinate in a child. It cannot restrict this fresh coordinate. If the query descends into that child, the prepared item is therefore still wide. Each queried item can require preparation at most once. The selection test ensures narrowness. Deleting vertices and admitting isolated vertices preserve the degree and height bounds. □

Lemma 5.5 (Graph bounds and termination)**.** *Suppose a call starts with degree less than k and height less than H, and its selected answer leaves positive volume. Then degree is at most k and height at most $2H$ throughout the call. At its end, every graph again satisfies the strict bounds. Restoration performs at most n refinements, all at the node changed by the answer.*

Proof. Preparation preserves the strict bounds by Lemma 5.4. The answer adds at most one arc, raising any degree by at most one. Any new directed path consists of two old paths joined by that arc, so it has fewer than $2H$ vertices. A positive-volume answer cannot create a directed cycle. An interval change adds no arcs.

Every refinement removes a vertex and its incident arcs from the parent. Thus degrees and heights cannot increase during restoration. The first pass removes every degree violation. The second pass removes every path violation without increasing degrees. All selected path vertices are processed, even if earlier refinements have broken the original path. Each refinement removes a different free item from this node, so there are at most n. A child receives an isolated coordinate and keeps all its old arcs. Its degree and height bounds are preserved as well, so no child requires restoration. □

Consequently a complete call makes at most $n+2$ refinements. During the call, every coordinate has at most $B = (k + 1)^{2H+1}$ other comparable coordinates in its active graph: all lie within undirected distance $2H$ in a graph of degree at most k. These are the bounds used by the local estimator and the anchor rules.

Proposition 5.6 (Legality)**.** *Every reached state has positive volume and represents only orders consistent with the public transcript. After every complete call, all nodes satisfy the strict degree and height bounds.*

Proof. We induct on comparison calls and, within a call, on refinements. Initially the root region is the unit cube, every order is represented, and the graph is empty. Suppose the claims hold at the start of a call.

Preparation refines only narrow coordinates. Its central dyadic rule chooses an interior position, so Lemma 5.2 preserves positivity and the order invariant. Lemma 5.4 shows that preparation terminates with the width needed for a probability query, unless the query has already been decided by two anchors.

The local estimator of Section 6 now applies to the positive queried region with the strict graph bounds. Its chosen answer retains at least $1/2 - \eta > 0$ of the volume and makes that comparison true in every represented order. In particular, a new arc cannot create a directed cycle. An answer determined by anchors, interval bounds, or a directed path was already true in every represented order and leaves the state unchanged. The selected answer is recorded in the public graph before any restoration refinement.

Lemma 5.5 gives degree at most k and height at most $2H$ during restoration. At each refinement these bounds and positivity allow the position guarantees of Section 6 to be used. The chosen anchor is interior, and Lemma 5.2 preserves the invariant and positivity for the next step. Restoration terminates with the strict bounds. This completes the induction. The numerical results used here concern arbitrary positive-volume regions defined by interval bounds and the inequalities of a DAG, subject to their stated hypotheses; their proofs do not assume legality of this adversary. □

6 Local probabilities and the choice of an anchor

To answer a comparison, the adversary estimates the fraction of feasible assignments supporting one answer. To refine an item, it chooses a parent position at which fixing that item retains enough volume. Both calculations use the same deterministic probability estimate, which we describe first.

6.1 Estimating a probability by a finite average

Fix the node handling the calculation, and let K be its region of feasible positions. The requested event is either $X_x < c$ for a dyadic threshold c, or $X_x < X_y$. All interval bounds have been tightened using the full DAG. The calculation uses a temporary vector Y of dyadic numbers, initialized at the effective lower endpoints. It visits the vertices in a fixed topological order, updating one number at a time. We call one such pass a *sweep*.

At the visit to x, compute the interval permitted by its current neighbors:

$$L_x = \max\bigl(\{\ell_x\} \cup \{Y_z : z \to x\}\bigr), \qquad U_x = \min\bigl(\{r_x\} \cup \{Y_z : x \to z\}\bigr). \tag{7}$$

Choose a number u from the finite set

$$\mathcal{U}_S = \left\{\frac{1}{2S}, \frac{3}{2S}, \ldots, \frac{2S-1}{2S}\right\}, \tag{8}$$

and replace Y_x by $(1-u)L_x + uU_x$. Here S is a power of two. The same update can be applied repeatedly, because the tightened lower endpoints satisfy all arc inequalities weakly, and every update preserves these weak inequalities. The interval $[L_x, U_x]$ can have length zero; the update then leaves Y_x at its sole permitted value.

The adversary enumerates every assignment of mesh values to a prescribed number of updates. Its estimate is the fraction of these computations whose final vector satisfies the requested event. This definition gives a deterministic calculation using maxima, minima, and dyadic arithmetic.

6.2 How much of the graph to retain

Suppose the full DAG has maximum degree at most d and every directed path has at most h vertices. Degree counts neighbors without regard to arc direction. Fix the required error ε and the width bound $1/R$. The following choices specify all sizes in the finite calculation:

$$M = (d+1)^{h+1}, \qquad C = M + 1, \qquad p = (4M^2)^{h+1}, \tag{9}$$

$$\delta = \frac{\varepsilon}{32CR}, \qquad \lambda = \lceil \log(1/\delta) \rceil, \qquad \mu = \lceil \log(8/\varepsilon) \rceil, \tag{10}$$

$$t = p\lambda + \mu, \qquad \rho = ht. \tag{11}$$

Take S to be the least power of two at least $\max\{2, ht/(2\delta)\}$. These integers can be computed using rational arithmetic and comparisons with powers of two. The proof in Appendix B explains their roles: t controls the number of sweeps, S controls the rounding error, and ρ includes every vertex that can influence a queried coordinate during those sweeps.

Retain all vertices within undirected distance ρ of a queried coordinate, together with all arcs between them. For a pair query, take the union of the two neighborhoods. Copy their effective endpoints from the full graph. These endpoints retain the interval restrictions implied by vertices outside the neighborhood. The retained graph has at most

$$b = 2(d+1)^{\rho+1} \tag{12}$$

vertices. Write b' for its actual number of vertices. Fix a topological order of this graph, choosing by item label whenever several vertices are eligible.

If a directed path already decides a pair query, its probability is zero or one. For a threshold outside the queried effective interval, the probability is likewise known. In the remaining cases use the following finite calculation. It leaves the adversary's stored state unchanged.

Algorithm 4: `Average`: estimate one comparison probability.

Input: The retained DAG and its effective intervals, the integers t, S, and the requested event E.
Output: A rational estimate of the probability of E.
$N_E \leftarrow 0$
foreach *assignment of mesh values to the $b't$ updates* **do**
  $Y_x \leftarrow \ell_x$ for every retained vertex x
  for $i = 1, \dots, t$ **do**
    foreach *vertex x in the fixed topological order* **do**
      Compute L_x, U_x from (7)
      $u \leftarrow$ the mesh value assigned to this update
      $Y_x \leftarrow (1-u)L_x + uU_x$
  if *E holds for the final vector Y* **then**
    $N_E \leftarrow N_E + 1$
return $N_E / S^{b't}$

Theorem 6.1 (Finite averaging)**.** *Let K have positive volume and be defined by dyadic interval bounds and the arc inequalities of a DAG with degree at most $d \geq 1$ and directed paths of at most $h \geq 1$ vertices. Let $R \geq 1$ and $0 < \varepsilon < 1/4$ be rational. Use the globally tightened endpoints and the parameters above. Under the uniform distribution on K:*

1. *If $W_x \geq 1/R$, the estimate of $\mathbb{P}(X_x < c)$ has error at most ε for every dyadic threshold c.*
2. *If at least one of W_x, W_y is at least $1/R$, the estimate of $\mathbb{P}(X_x < X_y)$ has error at most ε.*

The calculation enumerates $S^{b't} \leq S^{bt}$ sequences of updates, each with $b't$ coordinate updates. Apart from finding the neighborhood and handling the input endpoints, its cost depends only on d, h, R, ε.

The proof compares the finite calculation with an assignment uniformly distributed on the full region. Updating a coordinate by a uniform point in its currently permitted interval preserves that distribution. When both this assignment and a vector started at the lower endpoints use the same update choices, their queried coordinates approach one another. Rounding the choices to the finite mesh adds a controlled error. After t sweeps the queried values depend only on vertices within distance ht, which justifies restricting the calculation to the retained graph.

The width assumption controls the probability of an almost tied comparison. If x has at most M other comparable vertices, and y is incomparable with it, Corollary A.2 gives

$$\mathbb{P}(|X_x - X_y| \leq a) \leq \frac{2(M+1)a}{W_x}.$$

The corresponding bound also holds for a fixed threshold. Thus a wide queried coordinate converts a bound on coordinate error into a bound on comparison error. Appendix B proves the theorem, and Appendix C bounds the exact arithmetic.

Finite computations can end with equal coordinates. For instance, one sweep on two independent unit intervals with $S = 4$ gives frequencies $3/8$ for each strict order and $1/4$ for equality. Accordingly, the comparison rule uses the estimate $\widehat{p}$ for one strict event and $1 - \widehat{p}$ for the opposite answer. Since the continuous probabilities sum to one, this preserves the accuracy guarantee for both answers.

For the adversary, use $d = k$, $h = 2H$, and $\varepsilon = \eta$. Lemmas 5.5 and 5.4 supply the graph and width bounds whenever the estimator is called.

6.3 Density and the choice of a position

Fix a free coordinate x in a group of volume V. If $S_x(t)$ is the volume of the section obtained by setting $X_x = t$, its density is

$$f_x(t) = \frac{S_x(t)}{V}.$$

It measures volume per unit coordinate length: the fraction of assignments with $a < X_x < b$ is $\int_a^b f_x(t)\,dt$. By Lemma 5.3, fixing x at c and giving it a fresh unit coordinate in a child multiplies Z by $f_x(c)$. We therefore want a large density at the chosen anchor.

The two anchor rules obtain this in different ways. For a wide item we can estimate its cumulative distribution F_x and search for a position with substantial mass on either side. For a narrow item we use its interval endpoints alone. The following density estimates justify both rules.

Suppose x has effective interval (ℓ, r) of width W and at most B other comparable labels. Then

$$f_x(t) \le \frac{B+1}{W}, \qquad f_x(t) \ge \frac{4^{-B}}{W} \quad \left(\ell + \frac{W}{4} \le t \le r - \frac{W}{4}\right). \tag{13}$$

The lower bound gives an endpoint-only choice: every point in the central half has this much density. The upper bound limits the probability in a short interval; it will ensure that the midpoint search cannot continue halving its search interval indefinitely.

For the search we also need a density bound that improves as its cell shrinks. For any $a \le c \le b$,

$$(b-a)f_x(c) \ge \min\{F_x(c), 1 - F_x(c)\}\big(F_x(b) - F_x(a)\big). \tag{14}$$

If both tails at c and the mass of $[a, b]$ are bounded below, this forces the density at c to be a constant times $1/(b-a)$. Appendix A proves (13) and (14) by explicit maps between sections.

6.4 The midpoint search

Recall that the wide-item search starts with $[a, b] = [0, 1]$ and tests its midpoint c. It keeps the right half if the reported cumulative probability is below $1/4$, keeps the left half if it is above $3/4$, and otherwise accepts c. This cell records where to test next. Every test uses the same feasible region K and distribution F_x. Each test applies the finite estimator to the event $X_x < c$, using the same tightened endpoints and the same retained neighborhood. Only the threshold in the final event test changes.

Lemma 6.2 (Midpoint search)**.** *The midpoint search for a wide item terminates at an interior anchor c. If its accepted cell has length 2^{-m}, then*

$$f_x(c) \ge 2^{m-4}.$$

The search makes at most $\lceil \log(4(B+1)/W_x) \rceil + 1$ tests.

Proof. Every estimate has error at most $\eta < 1/32$. A test reported below $1/4$ has true value below $9/32$, and its midpoint becomes the new left endpoint. A test reported above $3/4$ has true value above $23/32$, and its midpoint becomes the new right endpoint. Since the initial endpoint values are $F_x(0) = 0$ and $F_x(1) = 1$, the search maintains

$$F_x(a) < 9/32, \qquad F_x(b) > 23/32.$$

At acceptance, the report lies in $[1/4, 3/4]$, so

$$\frac{7}{32} \le F_x(c) \le \frac{25}{32}, \qquad F_x(b) - F_x(a) \ge \frac{7}{16}.$$

The first bounds put c strictly inside its support. Applying (14) to the accepted cell gives

$$2^{-m} f_x(c) \geq \frac{7}{32}\frac{7}{16} = \frac{49}{512} > \frac{1}{16},$$

which proves the density guarantee.

For termination, the upper density bound in (13) gives, at every cell reached,

$$\frac{7}{16} \leq F_x(b) - F_x(a) \leq \frac{B+1}{W_x}\, 2^{-m}.$$

At $m \geq \lceil \log(4(B+1)/W_x) \rceil$, the upper bound would be at most $1/4 < 7/16$. Starting at $m = 0$ and increasing it by one at each rejection therefore permits at most the stated number of tests. □

6.5 A common bound for both anchor rules

The lower-bound proof will compare the logarithmic volume gain from refinement with the anchor's level m. The wide-item search gives $\log f_x(c) \geq m - 4$. The narrow-item rule gives the weaker bound $m - 3 - 2B$, but the choice $R = 2^{2kB}$ forces $m > 2kB$ whenever that rule is used. Thus its additional loss $2B$ is less than the fraction m/k of the level. Both choices satisfy the following bound.

Lemma 6.3 (Refinement density)**.** *Every anchor chosen by the adversary is interior and satisfies*

$$\log f_x(c) \geq (1 - 1/k)m - 4. \tag{15}$$

For a narrow item its level also satisfies

$$m > 2kB + 2. \tag{16}$$

Proof. For a wide item, Lemma 6.2 gives the stronger lower bound $m - 4$. For a narrow item, write $b = 2^{-m}$ for the selected cell length, so $W_x/8 < b \leq W_x/4$. The chosen midpoint is at distance at most $b/2 \leq W_x/8$ from the interval center. It belongs to the central half and is therefore interior. By (13) and $W_x < 8b$,

$$f_x(c) \geq \frac{4^{-B}}{W_x} > 2^{m-3-2B}.$$

Moreover, $b \leq W_x/4 < 1/(4R)$, so $m > \log R + 2 = 2kB + 2$. In particular $2B < m/k$, and hence $m - 3 - 2B > (1 - 1/k)m - 4$, proving (15). □

The narrow rule must use its prescribed fine grid even if a coarser dyadic midpoint lies inside the interval. Its high level is what allows the weaker central density estimate to satisfy the same bound as the wide-item search.

7 A geometric bound at termination

A correct sorter can finish only when the public answers determine a single order. We want to turn that fact into an upper bound on the volume of the final state. An item's position orders it within the node storing it; its place in the full order also depends on the anchors above that node. Consider a feasible assignment in which A is free at position $3/4$ in the root, while B has position $7/8$ inside a child anchored at $1/4$. The represented order is $B < A$, although the two raw coordinates satisfy $3/4 < 7/8$. Applying the usual $1/n!$ volume bound requires coordinates whose numerical order agrees with the full item order.

We therefore put all items onto one common line. Replace each anchor by a small interval, and place the child's recursively constructed line inside it. The order of items on the resulting line is exactly their represented order. Each item now has a coordinate in the same $(0, 1)$, so at termination all resulting assignments lie in one of the $n!$ equal-volume order regions. The only remaining question is how much this placement shrinks volume.

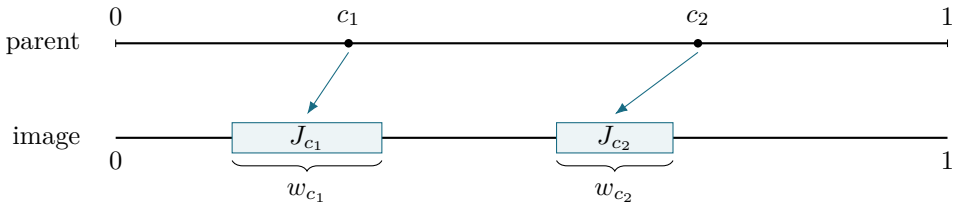


Figure 8: The packing map has slope 1/2 between anchors. Its jump at each anchor reserves an interval for the entire child subtree. The widths are shown schematically; their values are given by (17).

7.1 An interval for each child

Every group may eventually use anchors at arbitrarily fine dyadic levels. At level m there are 2^m possible anchors. Giving each one width proportional to 2^{-m} would use the same total width at every level and would not fit in a bounded interval. We make the widths decrease a little faster, by an extra factor $2^{-m/k}$. This makes their total finite while adding only m/k to the logarithmic cost of a child placement.

Specifically, reserve for each anchor at level m an interval of width

$$w(m) = \frac{2^{-(1+1/k)m}}{4k}. \tag{17}$$

Only children currently present receive intervals. The sum over all possible anchors below is a bound on every such finite set. The factor $4k$ leaves at least half the line for free positions. Indeed, at every node

$$\sum_c w_c \le \frac{1}{4k} \sum_{m \ge 0} 2^{-m/k} = \frac{1}{4k(1 - 2^{-1/k})} \le \frac{1}{2}. \tag{18}$$

For the last inequality, convexity gives $2^{-t} \le 1 - t/2$ for $0 \le t \le 1$, and we take $t = 1/k$.

At a node, map a free position t distinct from all anchors to

$$G(t) = \frac{t}{2} + \sum_{c<t} w_c. \tag{19}$$

The jump at an anchor c leaves the open interval

$$J_c = \left(\frac{c}{2} + \sum_{a<c} w_a, \frac{c}{2} + \sum_{a<c} w_a + w_c \right).$$

Place the recursively mapped child state inside J_c, using an affine map from $(0, 1)$ to J_c. The intervals J_c are disjoint, occur in anchor order, and fit in $(0, 1)$ by (18). Free positions stay in the gaps between them. All these widths are used only in the proof.

An item is scaled by a child interval's width every time its route from the root passes through that child. For example, an item at depth two has final position

$$a_1 + w_1(a_2 + w_2 G_v(X_x)),$$

where a_i is the left endpoint of the chosen child interval, measured in its own parent's unit interval, and G_v is the map in the group where the item is free. Its scale factor is $w_1 w_2/2$. The factor $1/2$ occurs once, at its free coordinate; the ancestor groups contribute only their child widths.

To multiply these scale factors over all items, let s_c be the number of original labels in the subtree below a parent–child edge c. The width w_c scales exactly these s_c items, so its contribution is $w_c^{s_c}$. The product of all child-width contributions is

$$\mathcal{W} = \prod_{\text{parent–child edges } c} w_c^{s_c}. \tag{20}$$

A comparison does not change $\mathcal{W}$. A refinement into a child of width w multiplies it by w: the refined label crosses that edge, while its membership in every other subtree remains unchanged. This holds for both a new child and an existing child.

Lemma 7.1 (Packing volume)**.** *The recursive map is injective and preserves the represented order. Its image $\mathcal{I} \subseteq (0,1)^n$ has volume*

$$\mathrm{vol}(\mathcal{I}) = 2^{-n} Z\mathcal{W}. \tag{21}$$

Consequently, if the transcript determines a unique total order, then

$$n! Z\mathcal{W} \le 2^n. \tag{22}$$

Proof. The map G puts each free position on the correct side of every child interval. Recursion therefore preserves the order of the original labels. Anchors and widths are fixed within the state. The image coordinate of a label depends only on its one free coordinate, through a strictly increasing map, so the whole map is injective.

Partition the domain at the finitely many anchor hyperplanes. On each piece, the map of a label is affine. Its slope is $1/2$ at its free coordinate, multiplied by the widths of the child intervals containing that label along the path to the root. There are exactly n factors of $1/2$, one per free coordinate. An edge width w_c occurs s_c times. Multiplying these coordinate scale factors shows that every piece has its volume multiplied by $2^{-n}\mathcal{W}$. The domain, a product of the node regions, has volume Z. Injectivity lets us add the image volumes of the pieces, proving (21).

At completion, legality places the image in the region of the cube realizing the unique transcript order. The $n!$ coordinate-order regions have equal volume $1/n!$, since coordinate permutations preserve volume and equality hyperplanes have volume zero. Hence $2^{-n} Z\mathcal{W} \le 1/n!$. □

7.2 Why a long path must use fine anchors

The items on a directed path have already been compared, directly or through intermediate items. Two of them therefore cannot enter the same child. A long path needs many distinct anchors, but there are few anchors at low levels. The following estimate makes this observation quantitative. It will pay for refining an entire path in the next section.

Lemma 7.2 (Total level of a path batch)**.** *The H labels of a selected directed path enter distinct children of their parent. If their levels are $m_1, \dots, m_H$, then*

$$\sum_{i=1}^{H} m_i \ge H(k-2). \tag{23}$$

Proof. Every arc on the selected path records an answered comparison. The path's labels are therefore pairwise comparable in the public transcript, and remain so after arcs are deleted. By Lemma 5.2, two of these labels cannot be admitted to the same child. The procedure completes all $H = 2^k$ refinements in this parent, so it uses H distinct anchors.

There are exactly 2^m possible anchors of level m. The smallest possible total level therefore uses all anchors of levels $0, \dots, k-1$ and one anchor of level k. Its value is

$$\sum_{m=0}^{k-1} m2^m + k = (k-2)2^k + k + 2 \ge H(k-2).$$

Every other choice of H distinct anchors has at least this total level. □

8 The comparison lower bound

The quantity $n!Z\mathcal{W}$ starts at $n!$ and is at most 2^n when sorting finishes. Each comparison retains at least $1/2 - \eta$ of the volume, but refinements can also reduce this quantity. We adjust it so that every complete repair preserves or increases it. Comparisons must then account for the required overall decrease.

8.1 A potential with one coefficient

Keep the chosen parameter $j \geq 6$ fixed, and recall that $k = 2^j$. Let e count the active arcs over all tree nodes, and put

$$s = \frac{j+10}{k}, \qquad \Psi = n!Z\mathcal{W}\,(Z2^{-e})^s = n!Z^{1+s}\mathcal{W}\,2^{-se}. \tag{24}$$

The factor Z^s gives additional weight to the volume recovered by refinement. The factor 2^{-se} charges for storing an arc and returns that amount when the arc is deleted. The same coefficient s serves both purposes. These quantities are used in the proof; computing them is unnecessary for answering a query.

Initially $Z = \mathcal{W} = 1$ and $e = 0$, so $\Psi = n!$. At termination, Lemma 7.1, $Z \leq 1$, and $e \geq 0$ give

$$\Psi = (n!Z\mathcal{W})(Z2^{-e})^s \leq 2^n. \tag{25}$$

For $j \geq 6$, the coefficient satisfies $0 < s \leq 1/4$: its value at $j = 6$ is $1/4$, and increasing j decreases it.

Consider a refinement at level m that deletes r incident arcs. The volume identity, the density bound, and the child width give

Quantity	Change caused by the refinement
$\log Z$	at least $(1 - 1/k)m - 4$
$\log \mathcal{W}$	exactly $-(1 + 1/k)m - (j + 2)$
e	exactly $-r$

The second row uses $\log(4k) = j + 2$. Combining these changes gives the following bound, which we will apply to all three repair rules.

Lemma 8.1 (Refinement inequality)**.** *Set*

$$a = s - \frac{s+2}{k}, \qquad C_j = j + 6 + 4s. \tag{26}$$

A refinement at level m deleting r arcs satisfies

$$\log \Psi_{\text{new}} - \log \Psi_{\text{old}} \geq am + sr - C_j. \tag{27}$$

Moreover, $a > 0$.

Proof. The child receives an isolated coordinate, so exactly the r incident parent arcs are deleted. The change in $\log \Psi$ is at least

$$(1 + s)\big((1 - 1/k)m - 4\big) - (1 + 1/k)m - (j + 2) + sr.$$

Its coefficient of m is $s - (s+2)/k$, and its constant term is $-(j + 6 + 4s)$. Finally, $ks = j + 10 \geq 16 > s + 2$, so $a > 0$. □

8.2 Every complete repair increases the potential

Proposition 8.2 (Repair monotonicity)**.** *Each query-preparation refinement, each degree refinement, and each complete path batch increases* Ψ*.*

Proof. A degree refinement deletes at least k arcs. Since $m \geq 0$ and $a > 0$,

$$\Delta \log \Psi \geq sk - C_j = 4(1 - s) > 0.$$

A preparation refines a narrow item. Equation (16) gives $m > 2kB + 2 > 2k$, and hence

$$\Delta \log \Psi > 2ka - C_j = j + 10 - 6s > 0.$$

For a path batch, let m_i be the level chosen at its ith refinement. There are $H = 2^k$ refinements. Lemma 7.2 gives $\sum_i m_i \geq H(k-2)$. Summing (27) over the whole batch and dropping the nonnegative arc terms gives

$$\begin{aligned} \log \frac{\Psi_{\text{after}}}{\Psi_{\text{before}}} &\geq H\big(a(k-2) - C_j\big) \\ &= H\left(2 - 7s + \frac{2s+4}{k}\right) > H/4 > 0, \end{aligned}$$

where the last inequality uses $s \leq 1/4$. □

An individual refinement within a path batch may decrease the potential. The proposition applies to the completed batch, which is why `Restore` finishes its selected list before selecting another path or returning an answer.

8.3 The decrease caused by a comparison

The endpoint bounds require a net decrease of at least $\log(n!) - n$ in $\log \Psi$. Complete repairs increase this quantity. We now bound the amount by which a single comparison can decrease it.

Theorem 8.3 (Fixed-parameter comparison bound)**.** *For every integer* $j \geq 6$*, the adversary with* $k = 2^j$ *is deterministic and legal. If a correct sorter terminates after* q *calls, then*

$$n \log n \leq \left(1 + \frac{2j + 22}{2^j}\right) q + 3n. \tag{28}$$

Proof. The procedures and tie rules are deterministic. Legality is Proposition 5.6, and each call terminates by Lemmas 5.5 and 5.4.

Put $a_\eta = -\log(1/2 - \eta)$. An estimated answer retains at least $1/2 - \eta$ of its node volume, leaves $\mathcal{W}$ unchanged, and adds at most one active arc. Its decrease in $\log \Psi$ is therefore at most

$$A_j = (1 + s)a_\eta + s.$$

A directly determined answer leaves the represented region and the potential unchanged. Preparations and complete restorations increase it by Proposition 8.2. Summing over calls and using the final bound (25) yields

$$\log(n!) \leq A_j q + n. \tag{29}$$

For $0 \leq u \leq 1$, convexity gives $2^{-u} \leq 1 - u/2$. Taking $u = 1/k$ shows

$$2^{-1/k} \leq 1 - \frac{1}{2k} \leq 1 - \frac{1}{50k}.$$

Since $\eta = 1/(100k)$, it follows that $a_\eta \le 1 + 1/k$. Consequently

$$A_j \le 1 + 2s + \frac{1+s}{k} < 1 + \frac{2j+22}{2^j}. \tag{30}$$

Finally,

$$\log(n!) \ge \int_1^n \log x \, dx = n \log n - \frac{n}{\ln 2} + \frac{1}{\ln 2}.$$

Combining this estimate with (29), (30), and $1 + 1/\ln 2 < 3$ proves the theorem. □

9 A uniform polynomial-time implementation

For each fixed j, the estimator uses a finite number of coordinate updates and mesh choices. To obtain the leading constant one, we must let j grow with n while keeping this work polynomial. We bound both the computation of the parameters and the size of every subsequent finite average. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

9.1 Selecting the parameter

For $j = 6, 7, \ldots$, compute $k = 2^j$, the parameters in (4), and the estimator parameters in (9)–(12), using $d = k$, $h = 2H$, and $\varepsilon = \eta$. Write t_j, S_j, ρ_j, b_j for the resulting values, and put

$$E_j = S_j^{b_j t_j}.$$

Here b_j bounds the size of every query neighborhood, so E_j bounds the number of update sequences for every estimate made with parameter j.

Fix a program that writes these complete parameter tuples successively, including E_j, in ordinary binary notation with delimiters. On input size n, simulate it for n machine steps. Among its completed tuples, choose the largest j satisfying

$$E_j \le n. \tag{31}$$

Use that parameter for the entire interaction. If no tuple qualifies, answer according to the fixed label order. The simulation writes only $O(n)$ bits; parsing the completed tuples and checking the inequality therefore take polynomial time as well.

These restrictions allow the selected parameter to grow without bound. For each fixed j, let T_j be the finite time required to finish its tuple. Whenever $n \ge \max\{T_j, E_j\}$, that tuple is available and qualifies. The selected $j(n)$ is then at least j. Thus $j(n) \to \infty$.

9.2 The cost of a finite average

For a chosen tuple, abbreviate its parameters by b, t, S. The bound (31) gives

$$bt \log S \le \log n. \tag{32}$$

Since $S \ge 2$, every enumerated sequence contains at most $bt \le \log n$ updates. Scanning the retained neighbors to form each update interval costs a polynomial in $\log n$ arithmetic operations. There are at most n sequences to enumerate.

If the input endpoints and threshold have at most Λ fractional binary digits, Appendix C bounds the extra precision during a sequence by

$$bt \log(2S) \le 2bt \log S \le 2 \log n.$$

The event tally has $O(\log n)$ bits. Thus the arithmetic cost of an estimate is polynomial in n and Λ, with an exponent independent of the chosen parameter.

9.3 Bounding the stored precision

Every stored endpoint is 0, 1, or a previously chosen dyadic anchor. For a wide coordinate, Lemma 6.2 bounds the number of midpoint tests and the precision of the chosen anchor by

$$K_0 = \lceil \log(4(B+1)R) \rceil + 2.$$

In this application the estimator has $M = B$ and $\delta = \eta/[32(B+1)R]$. Its choice $t \geq \lceil \log(1/\delta) \rceil$ implies $K_0 \leq t + 2 = O(\log n)$. A wide refinement therefore makes $O(\log n)$ estimator calls and chooses an anchor with $O(\log n)$ fractional digits.

For a narrow coordinate, suppose all current endpoints have at most Λ fractional digits. A positive effective width is at least $2^{-\Lambda}$. The chosen dyadic cell has length $2^{-m} > W_x/8 \geq 2^{-\Lambda-3}$, so $m < \Lambda + 3$. Its midpoint has at most $m + 1 \leq \Lambda + 3$ fractional digits. A narrow refinement therefore adds at most three digits. Tightening takes maxima and minima of stored endpoints and introduces no new numbers.

After P refinements, these facts give

$$\Lambda \leq K_0 + 3P. \tag{33}$$

Lemmas 5.4 and 5.5 give $P \leq (n+2)q$ after q complete calls, with at most $n + 2$ further refinements during the next call. Each refinement creates at most one node, so the tree has at most $P + 1$ nodes.

9.4 Bounding the state for every query sequence

A sorter may repeat comparisons indefinitely. After

$$Q_{\text{stop}} = 2n \lceil \log(n+1) \rceil \tag{34}$$

complete calls, choose a total order extending the comparison history by topologically sorting the public comparison graph. Answer subsequent queries from the ranks in this fixed order and discard the recursive state. The graph records every previous answer, including directly determined answers, so the fixed order respects the full history.

The cutoff exceeds $n \log n$. A sorter reaching it has already made more comparisons than required by Theorem 8.3. Before the cutoff, $P = O(n^2 \log(n+1))$, and (33) gives

$$\Lambda = O(n^2 \log(n+1)).$$

The tree has polynomially many nodes. Every estimate enumerates at most n sequences, with $O(\log n)$ updates in each and operands of polynomial binary length. Ordinary integer addition, comparison, and multiplication therefore give a fixed polynomial bit bound for an estimate.

A call makes at most $n + 2$ refinements. Each wide refinement makes $O(\log n)$ estimates; a narrow refinement uses only its endpoints. The call also makes at most one estimate to select its answer. All remaining operations are polynomial graph computations: routing the query, propagating bounds, finding neighborhoods, testing reachability, and selecting a long directed path. Path lengths are computed in reverse topological order. If H exceeds the number of vertices, no path qualifies; otherwise the selected path has at most n vertices. The fixed label rules can be implemented by these same computations. Thus every call, including the one that chooses the final fixed order, uses one fixed polynomial number of bit operations in n.

Proof of Theorem 1.1. The bounded initialization, finite averaging procedure, and cutoff define a single deterministic polynomial-time adversary. For sufficiently large n a tuple qualifies, and the chosen parameter satisfies $j(n) \to \infty$. If sorting finishes before the cutoff, Theorem 8.3 gives

$$q \geq \frac{n \log n - 3n}{1 + (2j(n) + 22)/2^{j(n)}} = (1 - o(1)) n \log n.$$

If the cutoff is reached, the sorter has already made more than $n \log n$ comparisons. Legality holds before the cutoff and is preserved by choosing a fixed order consistent with every answer. This proves the theorem. □

A Coordinate densities

We prove the density bounds used to choose anchors and to turn coordinate approximation into probability approximation. Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \neq x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \mathrm{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\mathrm{vol}_N K}, \qquad F_x(t) = \int_{-\infty}^{t} f_x(s)\, ds.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.1 (Coordinate density bounds)**.** *Suppose x has at most B other comparable labels and effective interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \leq \frac{B+1}{W}, \tag{35}$$

$$f_x(t) \geq \frac{4^{-B}}{W} \qquad (\ell + W/4 \leq t \leq r - W/4). \tag{36}$$

These bounds also hold after fixing any set of labels incomparable with x, for every resulting slice of positive volume.

Proof. We may fix any specified set of coordinates incomparable with x before starting the argument. Measure all volumes in the remaining free coordinates, and abbreviate the section volume by $S(t)$.

First we construct a map from the section at t into the section at a larger value s. Raising x may force its descendants to rise, but no other coordinates need move. We compress $[t, r]$ into $[s, r]$ and leave values above r unchanged. More precisely, for $\ell < t < s < r$, put $\alpha = (r-s)/(r-t)$ and apply

$$\phi(z) = \begin{cases} s + \alpha(z - t), & t \leq z \leq r, \\ z, & z \geq r \end{cases}$$

to x and all its descendants. Every descendant y has value greater than t. Its effective upper endpoint satisfies $r_y \geq r_x = r$, by the definition of effective intervals. A moved coordinate never decreases; if it starts below r, it stays below r, and if it starts at or above r, it stays fixed. Its lower and upper interval bounds are therefore preserved. The function ϕ is strictly increasing, so it preserves the inequalities imposed by arcs between moved coordinates. Every arc crossing the boundary of this set must enter it: an arc leaving a descendant would lead to another descendant. Only its head coordinate can move, and that coordinate never decreases, so its inequality also remains strict.

This injective map sends the section at t into the section at s. To compare volumes, partition the source section according to which moved coordinates lie below r. On each piece the map is affine and its Jacobian determinant in the free coordinates is α^j, where $j \leq B$: the coordinate x

is fixed, and at most B other coordinates are moved. Since $0 < \alpha < 1$, this determinant is at least α^B. The images of distinct pieces are disjoint because the map is injective. Adding their volumes gives

$$S(s) \geq \left(\frac{r-s}{r-t}\right)^B S(t) \qquad (t < s). \tag{37}$$

For a smaller target value s, the construction is reversed: lower x and its ancestors, compress $[\ell, t]$ into $[\ell, s]$, and leave values below ℓ fixed. Thus, for $\ell < s < t < r$, use

$$\phi(z) = \begin{cases} z, & z \leq \ell, \\ s + \dfrac{s-\ell}{t-\ell}(z-t), & \ell \leq z \leq t. \end{cases}$$

on x and its ancestors. The effective lower endpoint of every ancestor is at most ℓ. No moved coordinate rises, and one that starts at or above ℓ remains there. Thus all intervals are preserved. The map is strictly increasing on the moved set, so it preserves the inequalities imposed by arcs within that set. Every arc crossing its boundary leaves the set. Only its tail coordinate can move, and that coordinate never increases, so these inequalities also remain strict. Its determinant on each piece is at least $((s-\ell)/(t-\ell))^B$. Consequently

$$S(s) \geq \left(\frac{s-\ell}{t-\ell}\right)^B S(t) \qquad (s < t). \tag{38}$$

Neither construction moves a coordinate incomparable with x, so both remain valid after the conditioning at the start of the proof. A region of positive volume has some section of positive volume. The two maps carry that section to every $t \in (\ell, r)$ with a positive volume factor. Consequently, after fixing incomparable coordinates, we normalize the density of x over the same effective interval (ℓ, r) of width W.

Normalize S to obtain f_x. Integrating the two inequalities on their respective sides of t gives

$$1 \geq f_x(t)\left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^B ds + \int_t^r \left(\frac{r-s}{r-t}\right)^B ds\right) = \frac{W f_x(t)}{B+1}.$$

This proves the upper bound. For the lower bound, the density integrates to one over an interval of length W, so some $t_0 \in (\ell, r)$ has $f_x(t_0) \geq 1/W$. Fix $s \in [\ell + W/4, r - W/4]$. If $t_0 < s$, the ratio in (37) is at least $(r-s)/W \geq 1/4$; if $s < t_0$, the ratio in (38) is at least $(s-\ell)/W \geq 1/4$. If $s = t_0$, no map is needed. In every case $f_x(s) \geq 4^{-B}/W$. □

Corollary A.2 (A bound for small differences)**.** *If y is incomparable with x, or Y is a fixed number, then for every fixed interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \leq \frac{(B+1)a}{W}, \tag{39}$$

where $Y = X_y$ in the first case.

Proof. In the first case, condition on X_y. Lemma A.1 bounds the conditional density of x by $(B+1)/W$. Integrating over the translate $I + X_y$ and then averaging over X_y proves the claim. For fixed Y, apply the unconditional density bound. □

Lemma A.3 (Balancing sections)**.** *Let f be a coordinate density, extended by zero outside its effective interval. For $u \leq t \leq v$,*

$$f(u)f(v) \leq f(t)f(u+v-t). \tag{40}$$

Proof. Write $S(s) = \mathrm{vol}_{N-1} K_x(s)$. It suffices to construct a volume-preserving injection

$$K_x(u) \times K_x(v) \longrightarrow K_x(u+v-t) \times K_x(t).$$

Both products are measured in their $2(N-1)$ free coordinates. If $t = u$, $t = v$, or $S(u)S(v) = 0$, the inequality is immediate. Otherwise suppose $u < t < v$, take a pair of feasible assignments p, q with $p_x = u$ and $q_x = v$, and put $\delta = t - u$.

We first translate the second assignment by setting $h_i = q_i - \delta$ for every i. Translation preserves all arc inequalities, so both p and h satisfy them. The intermediate assignment h need not satisfy the interval bounds. We shall swap selected coordinates of p and h, then undo the translation on the second assignment.

Which coordinates must be swapped together? For each i, the two available values are p_i and h_i. Join the endpoints of an arc $i \to j$ by an undirected edge exactly when

$$\max(p_i, h_i) \geq \min(p_j, h_j).$$

An edge means that choosing the two endpoint values independently could violate $X_i < X_j$. If there is no edge, either value at i is strictly less than either value at j, so every choice is safe. Figure 9 illustrates the two cases.

Along an edge, $h_i - p_i$ and $h_j - p_j$ have the same strict sign. To see this, if $h_i \geq p_i$ and $h_j \leq p_j$, then $\max(p_i, h_i) = h_i < h_j = \min(p_j, h_j)$, because h satisfies the inequality for the arc $i \to j$. If $h_i \leq p_i$ and $h_j \geq p_j$, the same contradiction follows from $p_i < p_j$. These two cases also exclude a zero difference at either endpoint. Let C be the connected component containing x. Since $h_x - p_x = v - t > 0$, we obtain $h_i > p_i$ for every $i \in C$.

Now define the output pair by

$$(p'_i, q'_i) = \begin{cases} (q_i - \delta,\ p_i + \delta), & i \in C, \\ (p_i, q_i), & i \notin C. \end{cases}$$

This is precisely the proposed swap followed by the inverse translation. At x it gives $p'_x = v - \delta = u + v - t$ and $q'_x = u + \delta = t$, as required.

We check feasibility in two steps. First, on C we have $p_i < q_i - \delta < q_i$ and $p_i < p_i + \delta < q_i$. Thus both new values lie between the original feasible values p_i and q_i, and so remain in the interval for coordinate i. Outside C there is no change. Second, consider the arc inequalities before adding δ back. For an arc with both endpoints in C, the two assignments have exchanged values at both endpoints, so its inequality remains strict in each. If neither endpoint is in C, their values are unchanged. An arc crossing the boundary of C has no corresponding edge in the auxiliary graph, because C is a component; any choice of its endpoint values is therefore safe. Finally, adding the same δ to every coordinate of the second assignment preserves all arc inequalities. Both output assignments are feasible.

To recover the input, subtract δ from q' and observe that for every coordinate

$$\{p'_i, q'_i - \delta\} = \{p_i, q_i - \delta\} = \{p_i, h_i\}.$$

These unordered pairs determine the auxiliary graph. We therefore recover the same component C, swap its coordinates once more, and add δ to the second assignment. This recovers (p, q), proving injectivity.

There are only finitely many possible components C. On the part of the source with a fixed C, the map exchanges selected pairs of free coordinates and adds constants. Its Jacobian determinant has absolute value one. The parts and their images are measurable, and injectivity makes the images disjoint. Adding their volumes proves

$$S(u)S(v) \leq S(t)S(u+v-t).$$

Dividing by the square of the total volume proves the claim. □

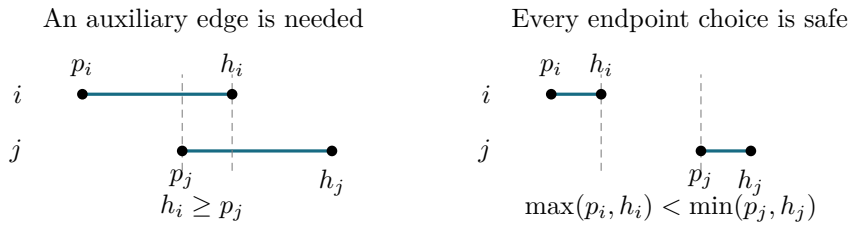


Figure 9: The auxiliary graph in Lemma A.3, for one arc $i \to j$. Each segment joins the two available values of its coordinate. On the left, swapping just one endpoint could select h_i and p_j, violating $X_i < X_j$. On the right, the two segments are separated in the required order, so swapping either endpoint is safe. Every boundary arc of a component has this second form.

Lemma A.4 (Density from mass on both sides)**.** *For a coordinate density f, its cumulative distribution F, and $a \leq c \leq b$,*

$$(b-a)f(c) \geq \min\{F(c), 1-F(c)\}\big(F(b)-F(a)\big). \tag{41}$$

Proof. Lemma A.3 gives $f(u)f(v) \leq f(c)f(u+v-c)$ for $u \leq c \leq v$. For fixed $v \geq c$, integration over $u \leq c$ gives

$$F(c)f(v) \leq f(c)\int_{-\infty}^{c} f(u+v-c)\,du \leq f(c),$$

because the integral of a density over any interval is at most one. Integrating this inequality over $c \leq v \leq b$ yields

$$F(c)\big(F(b)-F(c)\big) \leq (b-c)f(c).$$

Likewise, for fixed $u \leq c$, integration over $v \geq c$ gives $f(u)(1-F(c)) \leq f(c)$. Integrating over $a \leq u \leq c$ yields

$$\big(F(c)-F(a)\big)\big(1-F(c)\big) \leq (c-a)f(c).$$

Add these inequalities and replace the two tail factors on the left by their minimum. □

B Accuracy of finite averaging

We prove Theorem 6.1 by comparing the finite calculation with coordinate updates on the full feasible region K. First we use continuous uniform choices to show convergence from the lower endpoints. Then we bound the effect of replacing these choices by mesh values and of retaining only a neighborhood of the query. Throughout, $d, h, M, C, p, \delta, t, \rho, S$ have the meanings given in Section 6.

B.1 Comparing two sequences of coordinate updates

Start X uniformly distributed on K. Conditional on all coordinates other than X_x, its possible values form the interval

$$\big(\max(\{\ell_x\} \cup \{X_u : u \to x\}), \min(\{r_x\} \cup \{X_v : x \to v\})\big)\,.$$

The conditional density is constant. Replacing X_x by a uniform point in this interval therefore preserves the uniform distribution on K. This holds after every individual update and hence after every sweep.

Start a second vector Y at the tightened lower endpoints. Run both vectors on the full DAG in the same topological order, using the same fresh independent uniform number $U_x \in (0,1)$ at each update. This joint construction is a coupling of the two sequences. The vector X remains

uniformly distributed on K, while Y begins at a specified boundary point and approaches it under the shared updates.

Write $\Delta_x = |X_x - Y_x|$. Within a sweep, superscripts old and new mean the value at its start and after the coordinate's visit. Predecessors have already been updated, and successors retain their old values. The update rule therefore gives

$$\Delta_x^{\text{new}} \le (1 - U_x) \max_{u \to x} \Delta_u^{\text{new}} + U_x \max_{x \to v} \Delta_v^{\text{old}}, \tag{42}$$

where an empty maximum is zero. Indeed, the difference between the two lower endpoints is bounded by the largest predecessor discrepancy, and the difference between their upper endpoints by the largest successor discrepancy.

Let $B_h(x)$ be the vertices within undirected distance h of x. Starting at x, follow a predecessor attaining the maximum new discrepancy in (42), and repeat until reaching a source. This reverse path has $r \le h$ vertices. Every old successor value used along it belongs to $B_h(x)$. Put

$$D = \max_{z \in B_h(x)} \Delta_z^{\text{old}}.$$

At the source, the shared fixed lower endpoint contributes zero discrepancy. Substituting (42) backwards along the chosen path gives

$$\Delta_x^{\text{new}} \le \left(1 - \prod_{i=1}^{r} (1 - U_{z_i})\right) D.$$

Let A_x be the minimum of the fresh values $1 - U_z$ over x and all its ancestors. Each factor in this product is at least A_x, so

$$\Delta_x^{\text{new}} \le (1 - A_x^h) \max_{z \in B_h(x)} \Delta_z^{\text{old}}. \tag{43}$$

The maximizing path may depend on the fresh choices and discrepancies. Taking the minimum over the entire ancestor set makes the bound valid for every such path. Moreover, A_x depends only on fresh choices and is independent of the discrepancy vector at the start of the sweep.

B.2 A contraction bound independent of graph size

Both the ancestor set and $B_h(x)$ have at most $M = (d+1)^{h+1}$ vertices. Set $a_0 = 1/(4M^2)$. A union bound gives

$$\mathbb{P}(A_x < a_0) \le M a_0 = \frac{1}{4M}.$$

On the complementary event, the choice $p = (4M^2)^{h+1}$ gives

$$(1 - A_x^h)^p \le (1 - a_0^h)^p \le \frac{1}{1 + p a_0^h} = \frac{1}{1 + 4M^2} \le \frac{1}{4M}.$$

Here $(1-z)^{-p} \ge 1 + pz$ for $0 \le z < 1$ supplies the second inequality. Combining the two events yields $\mathbb{E}[(1 - A_x^h)^p] \le 1/(2M)$. Independence from the old discrepancies and (43) now imply

$$\begin{aligned} \mathbb{E}[(\Delta_x^{\text{new}})^p] &\le \frac{1}{2M} \sum_{z \in B_h(x)} \mathbb{E}[(\Delta_z^{\text{old}})^p] \\ &\le \frac{1}{2} \max_{z \in B_h(x)} \mathbb{E}[(\Delta_z^{\text{old}})^p]. \end{aligned} \tag{44}$$

Initially each discrepancy is at most one. Induction over sweeps gives $\mathbb{E}[\Delta_x(t)^p] \le 2^{-t}$ for every vertex. Thus

$$\mathbb{P}(\Delta_x(t) > \delta) \le 2^{-t} \delta^{-p} \le 2^{-\mu} \le \varepsilon/8, \tag{45}$$

using $t = p\lambda + \mu$, $2^\lambda \ge 1/\delta$, and $2^\mu \ge 8/\varepsilon$. The estimate holds separately at each queried vertex, with constants independent of the total number of vertices.

B.3 Replacing continuous choices by mesh values

Round each uniform choice U to the midpoint of its one of S equal subintervals of $(0,1)$. The rounded choices are independent and uniform on $\mathcal{U}_S$, and differ from the original choices by at most $1/(2S)$. Run a third vector $\widehat{Y}$ from the same lower endpoints, using these rounded choices on the full graph.

For a fixed current interval, changing the choice by at most $1/(2S)$ changes the updated coordinate by at most $1/(2S)$. Allowing the two current intervals to differ as well, let $e_x = |Y_x - \widehat{Y}_x|$. Then

$$e_x^{\text{new}} \le \max\left\{\max_{u\to x} e_u^{\text{new}}, \max_{x\to v} e_v^{\text{old}}\right\} + \frac{1}{2S},$$

with zero discrepancy at fixed endpoints. Expanding through the newly updated predecessors uses at most h vertices. One sweep therefore adds at most $h/(2S)$ to the maximum old discrepancy. Since both vectors start at the same point, after t sweeps

$$|Y_x(t) - \widehat{Y}_x(t)| \le \frac{ht}{2S} \le \delta \tag{46}$$

for every vertex and every sequence of choices.

B.4 Restricting the calculation to the query neighborhood

Expand one update backwards through the new predecessor values used in its sweep. A directed path contains at most h vertices, so this expansion uses at most $h-1$ predecessor edges. It may then use one successor edge to read a value from the start of the sweep. Consequently, the final value at x depends on old values within distance h and fresh choices within distance $h-1$.

After t sweeps, only initial values, endpoints, and choices within distance ht can influence x. At distance exactly ht, only initial values are needed. Keeping this neighborhood therefore reproduces the full rounded calculation exactly at x. For a pair query, keeping the union of both neighborhoods reproduces both values. The full graph's tightened endpoints are retained throughout, since they are inputs to every update.

One can use the restriction of a full topological order. Any other topological order of the retained graph gives the same result for fixed choices assigned to each vertex and sweep: updates of incomparable vertices commute, and adjacent exchanges of such vertices connect the topological orders. The neighborhood-size bound (12) follows from $1 + d + \cdots + d^\rho \le (d+1)^{\rho+1}$.

B.5 From coordinate error to comparison error

For an incomparable pair, choose a queried coordinate x with $W_x \ge 1/R$. There are at most M other vertices comparable with it. Corollary A.2 therefore gives

$$\mathbb{P}(|X_x - X_y| \le 4\delta) \le 8CR\delta = \varepsilon/4.$$

Outside the two exceptional events in (45), each continuous coordinate Y is within δ of its corresponding uniform coordinate X. Equation (46) adds at most another δ. If the comparisons made by X and $\widehat{Y}$ differ, either one of those exceptional events occurs or $|X_x - X_y| \le 4\delta$. Their total probability is at most $2(\varepsilon/8) + \varepsilon/4 = \varepsilon/2$.

For a threshold query there is one exceptional event. Otherwise, different answers require $|X_x - c| \le 2\delta$, which has probability at most $4CR\delta = \varepsilon/8$. The total disagreement probability is at most $\varepsilon/4$. A comparable pair is answered exactly by reachability, as specified in the estimator.

The difference between two event probabilities is bounded by the probability that their indicators disagree. The vector X remains uniform on K. Each possible sequence of rounded choices has probability $S^{-b't}$ on the retained graph, so its event probability is exactly the finite average in Algorithm 4. The preceding bounds prove Theorem 6.1.

C Exact arithmetic for the finite average

Let the retained graph have b' vertices, and suppose its dyadic endpoints and any query threshold have at most Λ fractional binary digits. A trajectory consists of $b't$ coordinate updates. Each update takes maxima and minima, followed by multiplication by mesh fractions with denominator $2S$. Charging a full denominator increase at every update bounds the precision of every intermediate coordinate by

$$\Lambda + b't \log(2S).$$

The values stay in $[0, 1]$, so their integer numerators have at most one additional bit. All comparisons and updates can therefore be carried out with exact integer arithmetic.

There are $S^{b't}$ trajectories. Enumerate their mesh choices as tuples of $b't$ integers in $\{0, \dots, S-1\}$. The event tally has at most $b't \log S + 1$ bits, and its denominator is the power of two $S^{b't}$. For a tally N and this denominator D, the comparison decision is $2N \geq D$. The midpoint search tests $4N < D$ and $4N > 3D$. These integer tests implement the probability rules exactly.

Section 9 chooses parameters satisfying $S^{bt} \leq n$. Hence $b't \log S \leq \log n$, the enumeration has at most n terms, and the extra coordinate precision is at most $2 \log n$ bits. These bounds, together with the bound on stored endpoint precision, give a polynomial bit cost for each estimate.

References

[1] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[3] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[6] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[7] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[8] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[9] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[10] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.